CPE 133 - 15, Fall 2025
Final Project

Description
This project involves a parking lot passcode entrance and occupancy counter. If the correct passcode is inputted, a car is let through the gate as long as a parking spot is open.

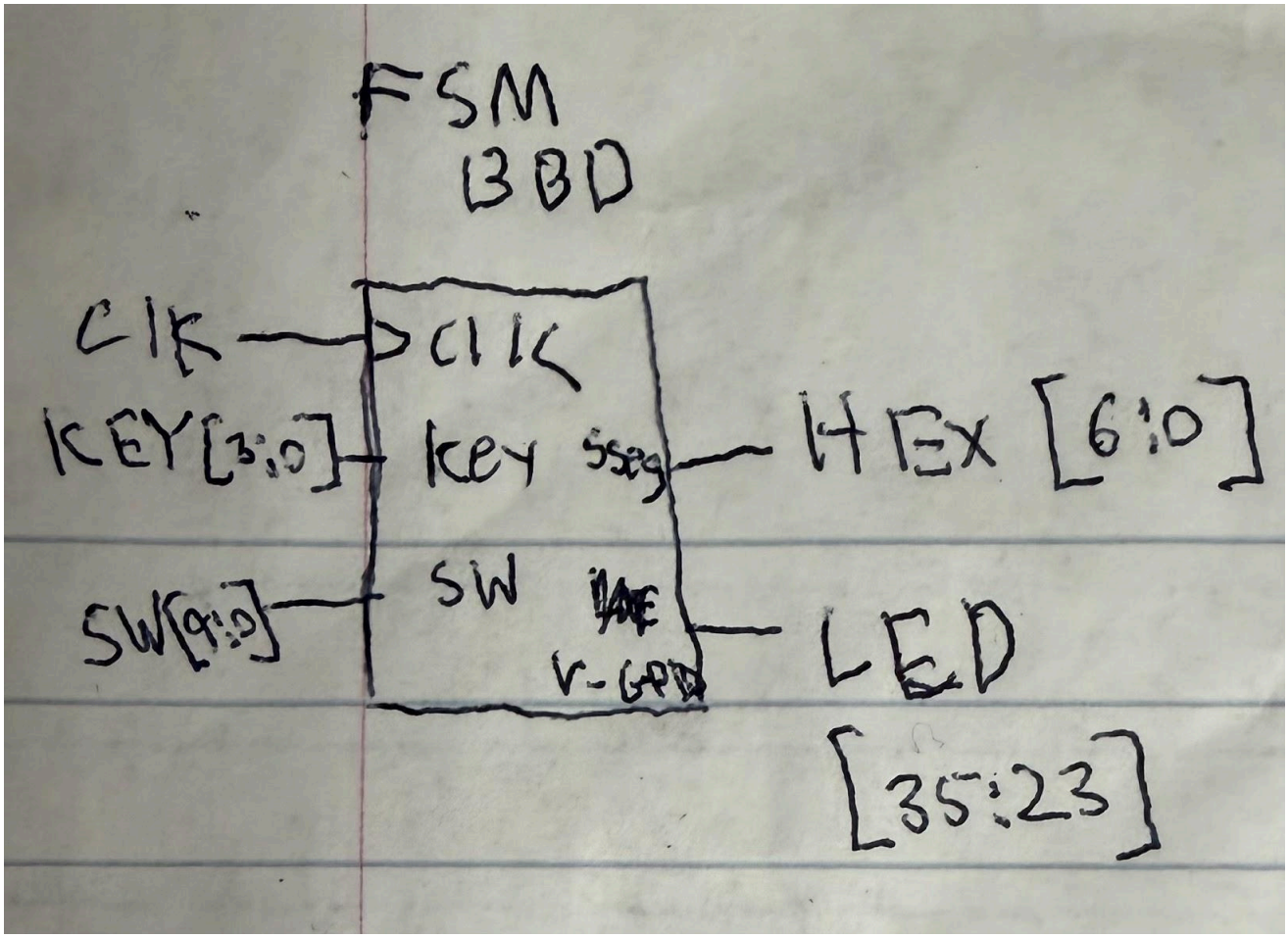


Figure 1: FSM Black Box Diagram

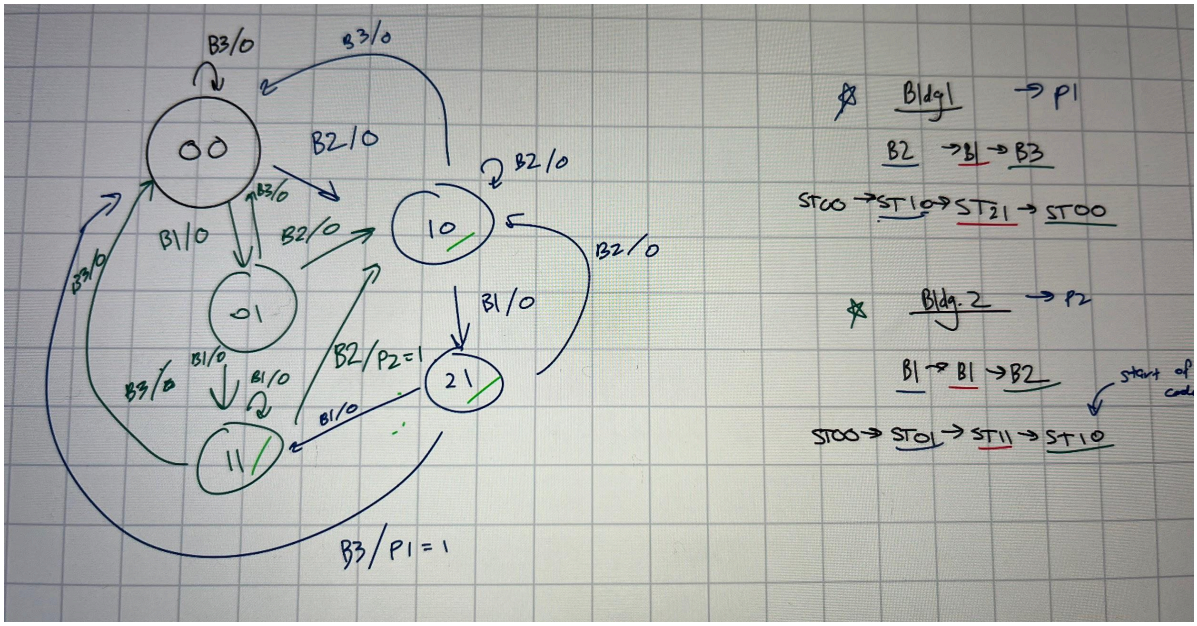


Figure 2: Finite State Transition Diagram

Circuit Schematic

FSM

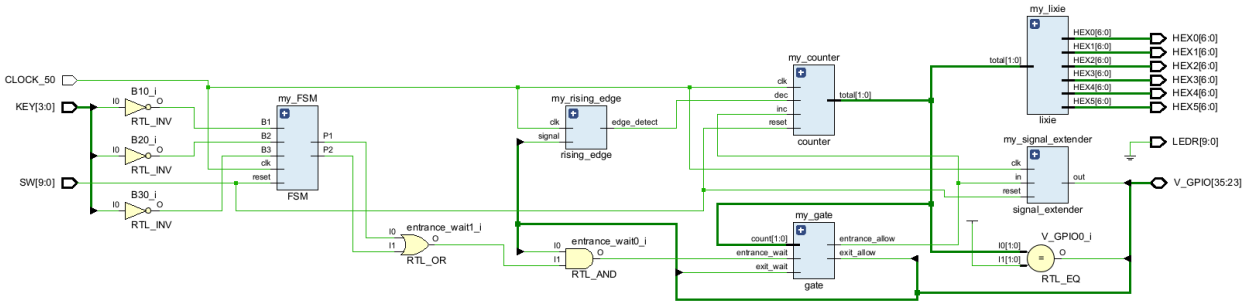


Figure 3: FSM Structural Model

Simulation

FSM

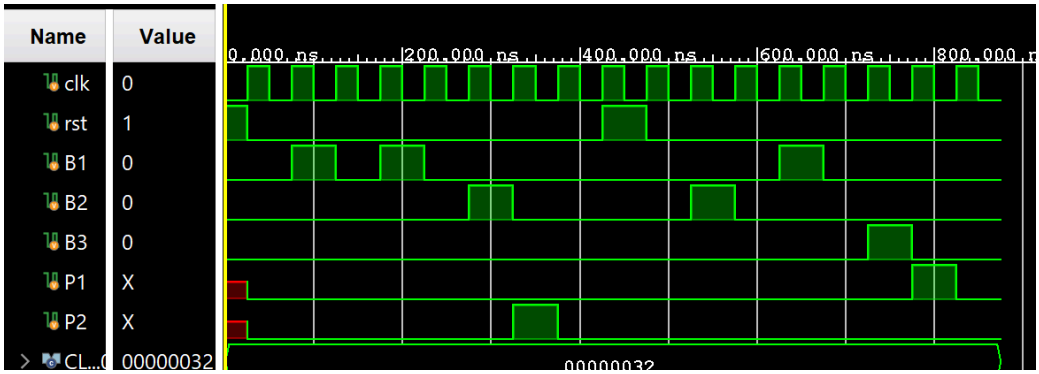


Figure 11: FSM Sim

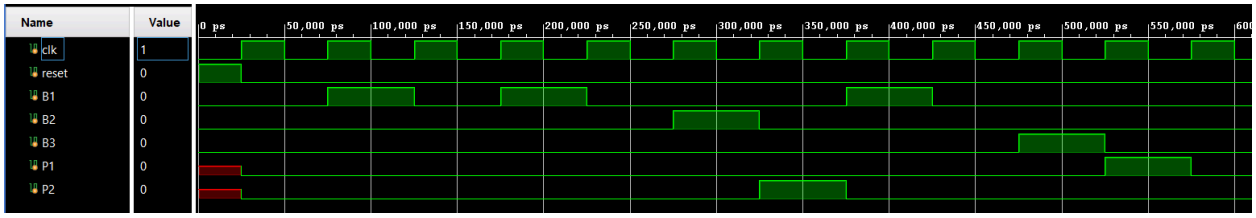


Figure 11: Overlapping FSM Sim

Counter

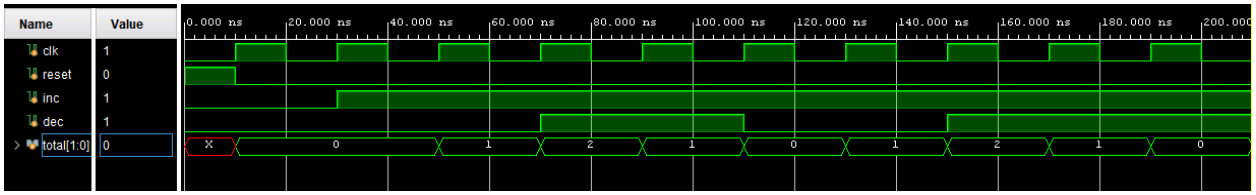


Figure 12: Counter Sim

Gate

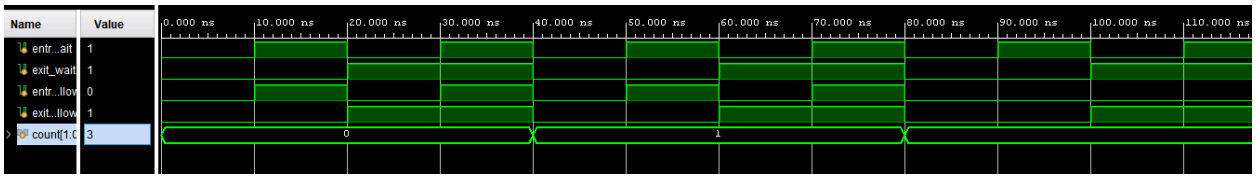


Figure 13: Gate Sim

The simulations produced the expected results.

Source Code

FSM

```
/*    @brief      An FSM that Keeps track of the number pad and detect
correct passcode
           entry.
      @param      B1, B2, B3: indicate the state of KEY[1], KEY[2], and
KEY[3],
              respectively.
      @return     P1, P2: one of them is asserted if one of the a correct
passcode is
              entered.
*/
module FSM (clk, reset, B1, B2, B3, P1, P2);
      input logic clk, reset, B1, B2, B3;
      output logic P1, P2;

      // TO DO
      // Declare state variables and build an FSM


parameter [2:0] ST00 = 3'b000, ST01 = 3'b001, ST10 = 3'b010, ST11 = 3'b011,
ST21 = 3'b100, ST5 = 3'b101, ST6 = 3'b110;

logic [2:0] PS = ST00;
logic [2:0] NS;


 always_ff @ (posedge clk, posedge reset)
    begin
        if (reset==1) PS <= ST00;
        else PS <= NS;
   end

   always_comb
   begin
   NS = PS;
   case(PS)

   ST00:
   begin
    if (B1) NS = ST01; //if input is B1 set NS to ST01
    else if (B2) NS = ST10;
    else if (B3) NS = ST00;
    end
```

```
ST10:
begin
if (B1) NS = ST21;
else if (B2) NS = ST10;
else if (B3) NS = ST00;
end

ST21:
begin
if (B1) NS = ST21;
else if (B2) NS = ST10;
else if (B3) NS = ST00;
else if (~B1&&~B2&&~B3) NS=ST5;
end

ST5:
begin
if (B1) NS=ST11;
else if (B2) NS=ST10;
else if (B3) NS=ST00;
end

ST01:
 begin
if (B1) NS = ST01;
else if (B2) NS = ST10;
else if (B3) NS = ST00;
else if (~B1&&~B2&&~B3) NS=ST6;
end

ST6:
begin
if (B1) NS=ST11;
else if (B2) NS=ST10;
else if (B3) NS=ST00;
end

ST11:
begin
if (B1) NS = ST11;
else if (B2) NS = ST10;
else if (B3) NS = ST00;
```

```
    end

    default: NS=ST00;
    endcase
    end

assign P1 = (PS == ST5 && B3);
assign P2 = (PS == ST11 && B2);




endmodule

module fsm_testbench();
      logic clk, reset, B1, B2, B3, P1, P2;

      FSM dut(.clk, .reset, .B1, .B2, .B3, .P1, .P2);

      parameter CLOCK_PERIOD = 20;

      initial begin
            clk <= 0;
            forever #(CLOCK_PERIOD/2) clk <= ~clk;
      end         // initial


      initial begin
            reset <= 1'b1;    B1 <= 1'b0; B2 <= 1'b0; B3 <= 1'b0;
@(posedge clk);
            reset <= 1'b0;
                                    @(posedge clk);
                                          B1 <= 1'b1; B2 <= 1'b0; B3 <=
1'b0;       @(posedge clk);
                                          B1 <= 1'b0; B2 <= 1'b0; B3 <=
1'b0;       @(posedge clk);
                                          B1 <= 1'b1;
                                    @(posedge clk);
                                          B1 <= 1'b0; B2 <= 1'b0; B3 <=
1'b0;       @(posedge clk);
                                                                  B2 <=
1'b1;                               @(posedge clk);
```

```
                                        B1 <= 1'b0; B2 <= 1'b0; B3 <=
1'b0;       @(posedge clk);
                                                                B1 <=
1'b1;                               @(posedge clk);

                                    B1 <= 1'b0; B2 <= 1'b0; B3 <= 1'b0;
      @(posedge clk);

                                                            B3 <= 1'b1;
                                @(posedge clk);


                                B1 <= 1'b0; B2 <= 1'b0; B3 <= 1'b0;
@(posedge clk);


                                                    @(posedge clk);
            $stop;

      end         // initial
endmodule
```

Counter

```
/*    @brief      Keep track of the number of cars currently in the parking
lot.
      @param      inc: increment the car counter by 1;
              dec: decrement the car counter by 1.
      @return     total(4-bit): number of cars currently in the parking
lot.
*/
module counter (
    input logic clk, reset, inc, dec,
    output logic [1:0] total
);

    always_ff @(posedge clk) begin
        if (reset) begin
        total <= 2'd0;
            // TO DO
        end
        else begin
```

```
            if (inc && !dec) total <= total + 2'd1; //Count up
            else if (dec) total <= total - 2'd1; //Count down
            // TO DO
        end
    end
endmodule

module counter_testbench();
    logic clk, reset, inc, dec;
    logic [1:0] total;

    counter dut(.clk, .reset, .inc, .dec, .total);

      // Configure clock
      parameter CLOCK_PARAMETER = 20;

      initial begin
            clk <= 0;
            // Forever toggles the clk
            forever #(CLOCK_PARAMETER/2)  clk <= ~clk;
      end         // initial

      // Simulation
      initial begin
            // Simulate a square wave with 4 cycles
            reset <= 1'b1;  inc <= 1'b0;    dec <= 1'b0;    @(posedge clk);
            reset <= 1'b0;                                  @(posedge clk);
                            inc <= 1'b1;                    @(posedge clk);
                                                            @(posedge clk);
                                            dec <= 1'b1;    @(posedge clk);
                            inc <= 1'b1;                    @(posedge clk);
                                            dec <= 1'b0;    @(posedge clk);
                                                            @(posedge clk);
                                            dec <= 1'b1;    @(posedge clk);
                                            dec <= 1'b1;    @(posedge clk);
                                            dec <= 1'b1;    @(posedge clk);
            $stop;
      end

endmodule
```

Gate

```
/*    @brief      Control the entrance and exit gates.
      @param      entrance_wait, exit_wait: indicate if a car is waiting to
                  enter/exit, respectively;
                  count(4-bit): number of cars currently in the parking
lot.
      @return     entrance_allow, exit_allow: allow a car to enter/exit.
*/
module gate(
            input  logic                  entrance_wait, exit_wait,
            input  logic [1:0]      count,
            output logic                  entrance_allow, exit_allow
);




      // TO DO
always_comb begin
      entrance_allow = (entrance_wait && (count < 2'd3));
      exit_allow = exit_wait;
      end

endmodule


/* @brief   Testbench for the above function.
      @param      None
      @return     None
*/
module gate_testbench();
      logic             entrance_wait, exit_wait, entrance_allow,
exit_allow;
      logic [1:0] count;

      // Initialization of a dummy gate module for simulation.
      // It takes entrance_wait, exit_wait, and count as inputs and
      // outputs entrance_allow and exit_allow.
      gate dut(.entrance_wait, .exit_wait, .count, .entrance_allow,
.exit_allow);

      // Simulation
      initial begin
            // Parking is empty
```

```
            count <= 2'd0;    entrance_wait <= 1'b0;  exit_wait <= 1'b0;
            #10;
                                        entrance_wait <= 1'b1;
                                        #10;
                                        entrance_wait <= 1'b0;  exit_wait
<= 1'b1;          #10;
                                        entrance_wait <= 1'b1;
                                        #10;


            // Parking is partially filled
            count <= 2'd1;    entrance_wait <= 1'b0;  exit_wait <= 1'b0;
            #10;
                                        entrance_wait <= 1'b1;
                                        #10;
                                        entrance_wait <= 1'b0;  exit_wait
<= 1'b1;          #10;
                                        entrance_wait <= 1'b1;
                                        #10;


            // Parking is fully filled
            count <= 2'd3;    entrance_wait <= 1'b0;  exit_wait <= 1'b0;
            #10;
                                        entrance_wait <= 1'b1;
                                        #10;
                                        entrance_wait <= 1'b0;  exit_wait
<= 1'b1;          #10;
                                        entrance_wait <= 1'b1;
                                        #10;

      end         // initial

endmodule
```